

ECE 5159-Class 5

DIGITAL SYSTEM DESIGN WITH VHDL



**Engr. Professor Alice
Oluwafunke Oke**

Department of Computer Engineering, ACU, Oyo

DESIGN OF A FULL ADDER

Cin	A	B	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Same as half
adder

↑
Carry in

↑ Sum ↑ Carry out

USING SUBCIRCUITS

$$\begin{aligned} S &= \overline{Cin} \overline{A} B + \overline{Cin} A \overline{B} + Cin \overline{A} \overline{B} + A B Cin \\ &= \overline{Cin} (\overline{A} B + A \overline{B}) + Cin (\overline{A} \overline{B} + A B) \\ &= \overline{Cin} (A \oplus B) + Cin (\overline{A \oplus B}) \end{aligned}$$

Let $A + B = X$

$$\begin{aligned} S &= \overline{Cin} X + Cin \overline{X} \\ &= Cin \oplus X \\ &= Cin \oplus (A \oplus B) \\ C &= \overline{Cin} A B + Cin \overline{A} B + Cin A \overline{B} + Cin A B \\ &= \overline{Cin} A B + Cin \overline{A} B + Cin A (\overline{B} + B) \\ &= \overline{Cin} A B + Cin \overline{A} B + Cin A \\ &= \overline{Cin} A B + Cin (A B + A) \longrightarrow A B + A = A + B \\ &= \overline{Cin} A B + Cin (A + B) \\ &= \overline{Cin} A B + Cin A + Cin B \\ &= (\overline{Cin} A + Cin) B + Cin A \\ &= B (A + Cin) + Cin A \\ &= A B + Cin B + Cin A \end{aligned}$$

USING SUBCIRCUITS

- A VHDL entity defined in one source code file can be used as a subcircuit in another source code file.
- In VHDL, subcircuit is called a component
- A subcircuit must be declared using a component declaration

COMPONENT fulladd

**PORT (Cin, x, y : IN STD_LOGIC;
s, Cout : OUT STD_LOGIC);**

END COMPONENT ;

DECLARING A COMPONENT IN A PACKAGE

```
LIBRARY ieee ;
```

```
USE ieee.std_logic_1164.all ;
```

```
PACKAGE fulladd_package IS
```

```
    COMPONENT fulladd
```

```
        PORT (Cin, x, y : IN STD_LOGIC;  
              s, Cout : OUT STD_LOGIC);
```

```
    END COMPONENT ;
```

```
END fulladd_package ;
```

CONCURRENT ASSIGNMENT STATEMENTS

- A Concurrent assignment statement is used to assign a value to a signal in an architecture body. E.g.

$$s \leq x \text{ XOR } y \text{ XOR } \text{Cin}$$

- VHDL provides four different types of concurrent assignment statements
 - Simple signal assignment
 - Selected signal assignment
 - Conditional signal assignment
 - Generate statements

CODE FOR A FULL ADDER

```
LIBRARY ieee ;
```

```
USE ieee.std_logic_1164.all ;
```

```
ENTITY fulladd IS
```

```
    PORT (Cin, x, y : IN STD_LOGIC;  
          s, Cout : OUT STD_LOGIC);
```

```
END fulladd ;
```

```
ARCHITECTURE Logicfunc OF fulladd IS  
BEGIN
```

```
    s <= x XOR y XOR Cin ;
```

```
    Cout <= (x AND y) OR (x AND Cin) OR (y AND Cin);
```

```
END Logicfunc ;
```

EXAMPLE OF CONCURRENT ASSIGNMENT STATEMENTS

Simple Signal assignment

```
LIBRARY ieee ;  
USE ieee.std_logic_1164.all ;
```

```
ENTITY fulladd IS  
    PORT (Cin, x, y : IN STD_LOGIC;  
          s, Cout : OUT STD_LOGIC);  
END fulladd ;
```

```
ARCHITECTURE Logicfunc OF fulladd IS  
BEGIN
```

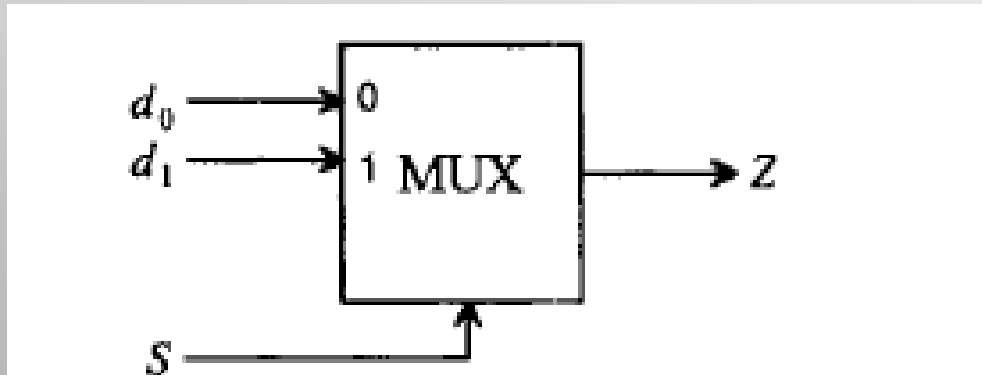
```
    s <= x XOR y XOR Cin;  
    Cout <= (x AND y) OR (x AND Cin) OR (y AND Cin);  
END Logicfunc ;
```


EXAMPLE OF CONCURRENT ASSIGNMENT STATEMENTS

Selected Signal assignment

Selected signal assignment allows a signal to be assigned one of several values based on a selection criterion.

Consider a multiplexer: a combinational circuit that selects one of n input lines and provides it on the output with the block diagram and Truth Table as shown



Truth Table of the 2-to-1 Multiplexer

S	d_0	d_1	Z
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

EXAMPLE OF CONCURRENT ASSIGNMENT STATEMENTS

The Selected Signal assignment for the multiplexer is as shown:

```
LIBRARY ieee ;  
USE ieee.std_logic_1164.all ;  
  
ENTITY mux2to1 IS  
    PORT (d0, d1, s : IN STD_LOGIC;  
          Z       : OUT STD_LOGIC);  
END mux2to1 ;  
  
ARCHITECTURE Behaviour OF mux2to1 IS  
BEGIN  
    WITH s SELECT  
        Z <= d0 WHEN '0';  
          d1 WHEN OTHERS;  
END Behaviour ;
```

EXAMPLE OF CONCURRENT ASSIGNMENT STATEMENTS

Conditional Signal assignment

Conditional signal assignment is similar to selected signal assignment. It allows a signal to be set to one of several values. An example using the mux2to1 is

```
LIBRARY ieee ;  
USE ieee.std_logic_1164.all ;
```

```
ENTITY mux2to1 IS  
    PORT (d0, d1, s : IN STD_LOGIC;  
          Z       : OUT STD_LOGIC);  
END mux2to1 ;
```

```
ARCHITECTURE Behaviour OF mux2to1 IS  
BEGIN
```

```
    Z <= d0 WHEN s = '0' ELSE d1;
```

```
END Behaviour ;
```

EXAMPLE OF CONCURRENT ASSIGNMENT STATEMENTS

Generate Statement

Generate statement is often used to describe regularly structured hierarchical code.

An example of such circuit is a 16-to-1 multiplexer using five instances of a 4-to-1 multiplexer subcircuit or a 4-bit ripple-carry adder using four instances of full adder.

This structure of a regular code for a 16-to-1 multiplexer can be written in a more compact form using a loop.

VHDL provides a feature called the FOR GENERATE statement for describing such regularly structured hierarchical code.

EXAMPLE OF N-RIPPLY ADDER CIRCUIT AND CODE

Consider this 3-input full adder

```
LIBRARY ieee ;
```

```
USE ieee.std_logic_1164.all ;
```

```
ENTITY fulladd IS
```

```
    PORT (Cin, x, y : IN STD_LOGIC;  
          s, Cout : OUT STD_LOGIC);
```

```
END fulladd ;
```

```
ARCHITECTURE Logicfunc OF fulladd IS
```

```
BEGIN
```

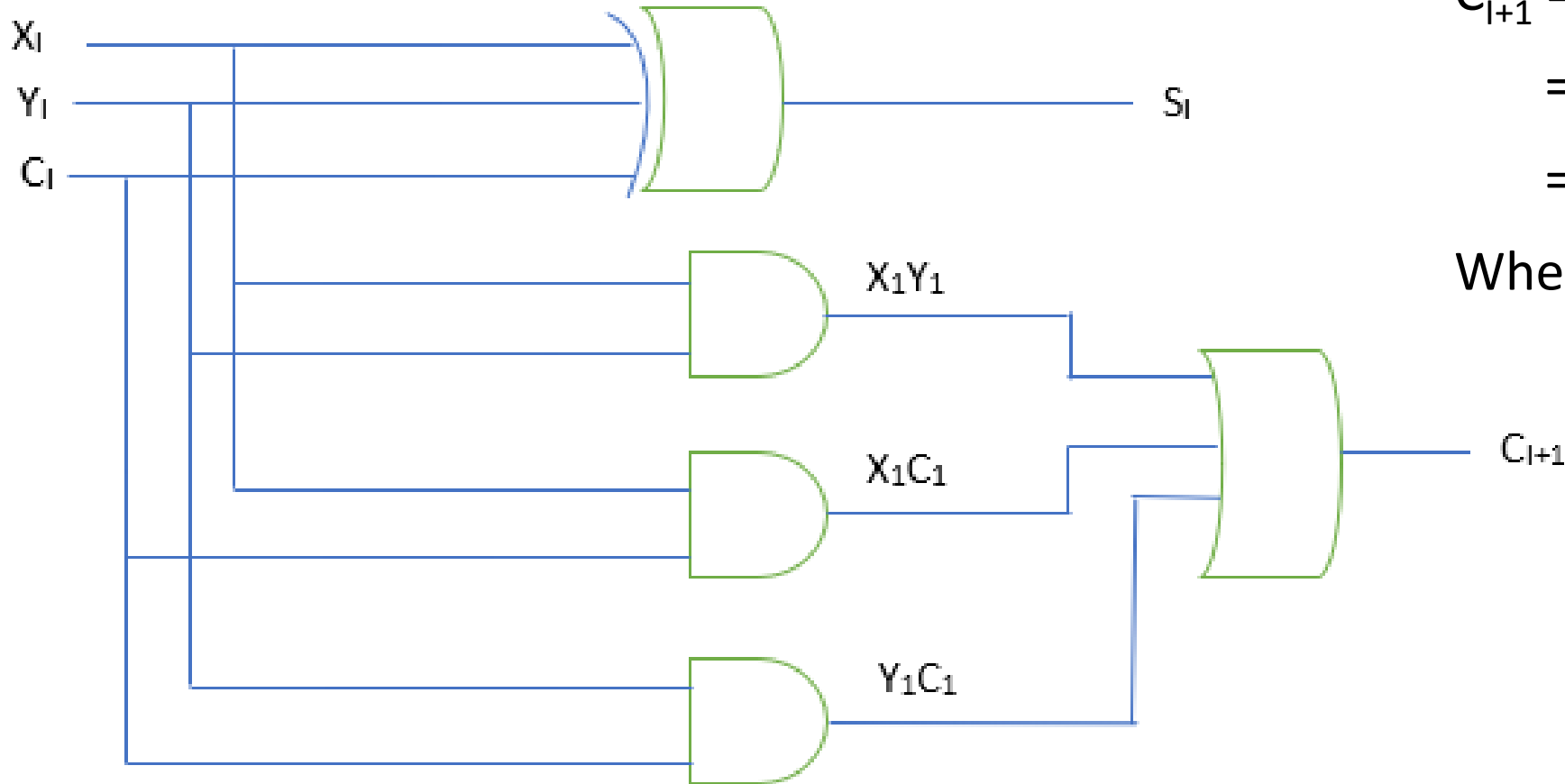
```
    s <= x XOR y XOR Cin ;
```

```
    Cout <= (x AND y) OR (x AND Cin) OR (y AND Cin);
```

```
END Logicfunc ;
```

EXAMPLE OF N-BIT RIPPLE CARRY ADDER CIRCUIT AND CODE

The circuit for the 3-input full adder



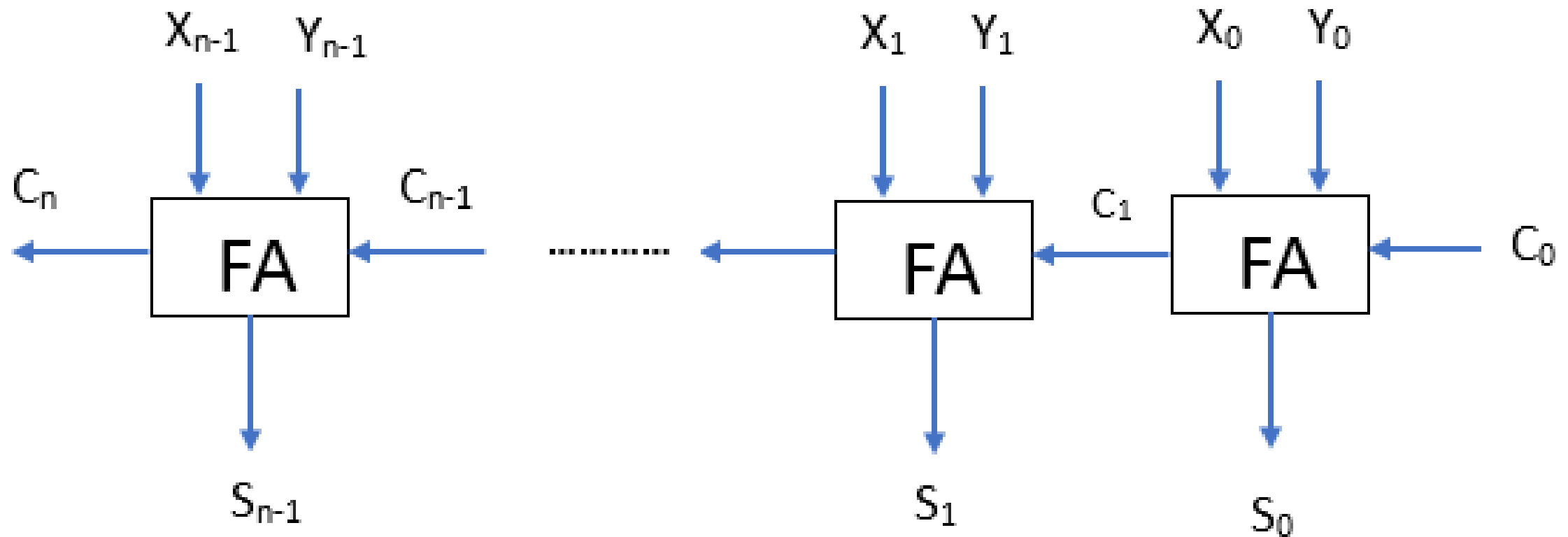
The Analysis

$$\begin{aligned}C_{1+1} &= X_1Y_1 + C_1X_1 + Y_1C_1 \\&= X_1Y_1 + C_1(X_1 + Y_1) \\&= g_1 + p_1c_1\end{aligned}$$

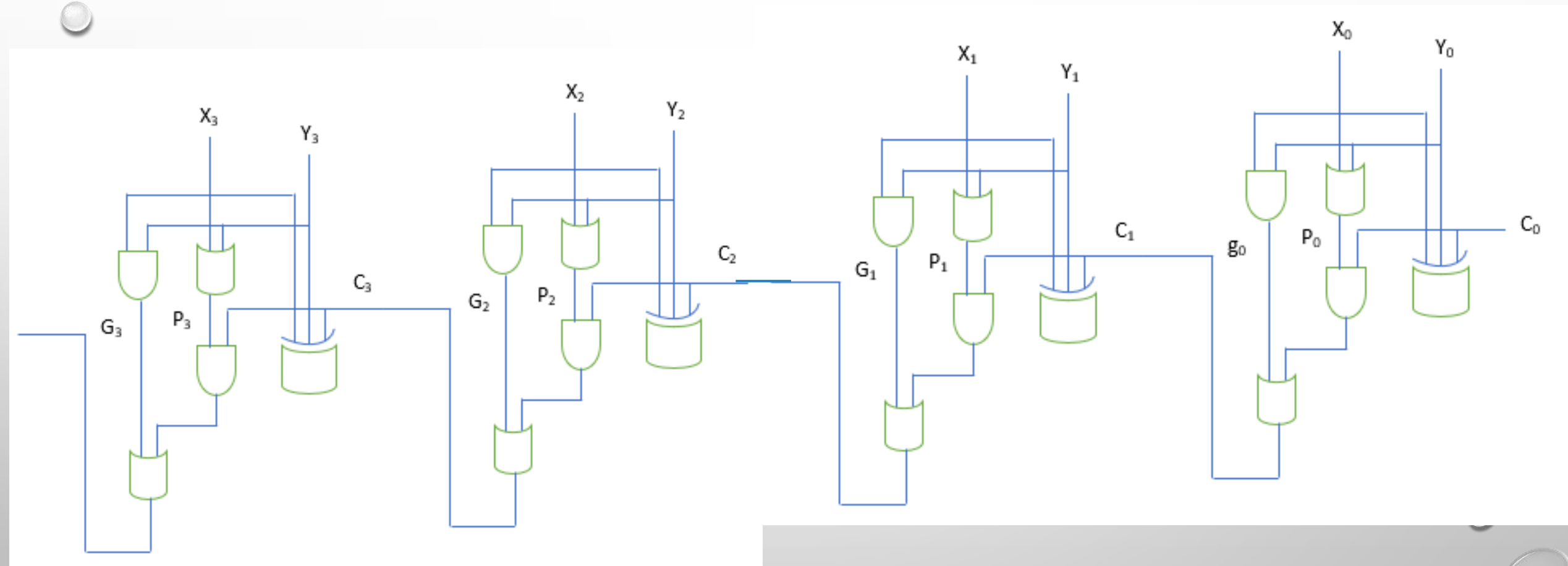
Where $g_1 = X_1Y_1$, and $p_1 = (X_1 + Y_1)$

EXAMPLE OF N-BIT RIPPLE CARRY ADDER CIRCUIT

The symbol of an n-bit ripple-carry full adder is presented



EXAMPLE OF N-BIT RIPPLE CARRY ADDER CIRCUIT



A 4-bit ripple-carry full adder

EXAMPLE OF N-BIT RIPPLE CARRY ADDER CODE

The VHDL code for a 4-bit ripple-carry full adder

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY adder4 IS
    PORT (Cin          : IN STD_LOGIC;
          x3, x2, x1, x0 : IN STD_LOGIC;
          y3, y2, y1, y0 : IN STD_LOGIC;
          s3, s2, s1, s0 : IN STD_LOGIC;
          Cout          : OUT STD_LOGIC);
END adder4 ;
ARCHITECTURE Structure OF adder4 IS
    SIGNAL c1, c2, c3 : STD_LOGIC;
    COMPONENT fulladd
        PORT (Cin x, y      : IN STD_LOGIC;
              s, Cout       : OUT STD_LOGIC);
    END COMPONENT;
BEGIN
    stage0: fulladd PORT MAP (Cin, x0, y0, s0, c1);
```

EXAMPLE OF N-BIT RIPPLE CARRY ADDER CODE Contd

```
stage1: fulladd PORT MAP (Cin, x1, y1, s1, c2);  
stage2: fulladd PORT MAP (Cin, x2, y2, s2, c3);  
stage3: fulladd PORT MAP (  
    Cin => c3, Cout => Cout, x => x3, y => y3, s => s3);  
END Structure ;
```

SEQUENTIAL ASSIGNMENT STATEMENTS

- Another type of assignment statement provided by VHDL is sequential assignment statements, the order of the statements in the code are executed sequentially. The order is significant in Sequential but not in Concurrent assignment statements.
- The semantics of the code can be affected if done otherwise
- The three variants of the sequential assignment statements are:
 - IF statement
 - CASE statement
 - Loop statement

PROCESS STATEMENT

- The PROCESS statement is used to separate sequential statements from concurrent statements.
- The PROCESS statement appears inside an architecture body and it encloses other statements within it.
- The IF, CASE, and LOOP statements can appear only inside a PROCESS.
- The general form of these statements are:

General Form of a PROCESS STATEMENT

```
[process_label;]  
PROCESS [(signal name {, signal name} )]  
    [VARIABLE declarations]  
BEGIN  
    [WAIT statement]  
    [Simple Signal Assignment Statements]  
    [Variable Assignment Statements]  
    [IF Statements]  
    [CASE Statements]  
    [LOOP Statements]  
END PROCESS [process_label] ;
```

General Form of an IF STATEMENT

```
IF expression THEN
    statement ;
    {statement ;}
ELSIF
    statement ;
    {statement ;}
ELSE
    statement ;
    {statement ;}
ENDIF;
```

E.g.

```
IF Sel = '0' THEN
    F <= x1 ;
ELSE
    F <= x2 ;
ENDIF;
```

SEL	X1	X2	F=MUL
0	0	0	0
0	0	1	0
1	1	0	0
1	1	1	1

EXAMPLE OF PROCESS STATEMENT USING IF-THEN-ELSE

PROCESS STATEMENT using if-then-else statement for 2-to-1 Multiplexer

```
LIBRARY ieee ;
```

```
USE ieee.std_logic_1164.all ;
```

```
ENTITY mux2to1 IS
```

```
    PORT (d0, d1, s : IN STD_LOGIC;  
          Z      : OUT STD_LOGIC);
```

```
END mux2to1 ;
```

```
ARCHITECTURE Behaviour OF mux2to1 IS
```

```
BEGIN
```

```
    PROCESS (d0, d1, s )
```

```
    BEGIN
```

```
        IF s = '0' THEN
```

```
            Z <= d0;
```

```
        ELSE
```

```
            Z <= d1;
```

```
        END IF ;
```

```
    END PROCESS ;
```

```
END Behaviour ;
```

ALTERNATIVE CODE FOR IF-THEN-ELSE STATEMENT

```
LIBRARY ieee ;  
USE ieee.std_logic_1164.all ;
```

```
ENTITY mux2to1 IS  
    PORT (d0, d1, s : IN STD_LOGIC;  
          Z      : OUT STD_LOGIC);  
END mux2to1 ;
```

```
ARCHITECTURE Behaviour OF mux2to1 IS  
BEGIN  
    PROCESS (d0, d1, s )  
    BEGIN  
        Z <= d0;  
        IF s = '1' THEN  
            Z <= d1;  
        END IF ;  
    END PROCESS ;  
END Behaviour ;
```


PROCESS STATEMENT WITH CASE STATEMENT

CASE STATEMENT

A case statement is similar to selected signal assignment in that the case statement has a selection signal and includes WHEN clauses for various valuations of this selection signal

The general form of a CASE statement and the **CASE STATEMENT** for 2-to-1 Multiplexer are presented next

General Form of a CASE STATEMENT

```
CASE expression IS  
    WHEN constant_value =>  
        statement ;  
    {statement ;}  
    WHEN constant_value =>  
        statement ;  
    {statement ;}  
    WHEN OTHERS =>  
        statement ;  
    {statement ;}  
END CASE ;
```

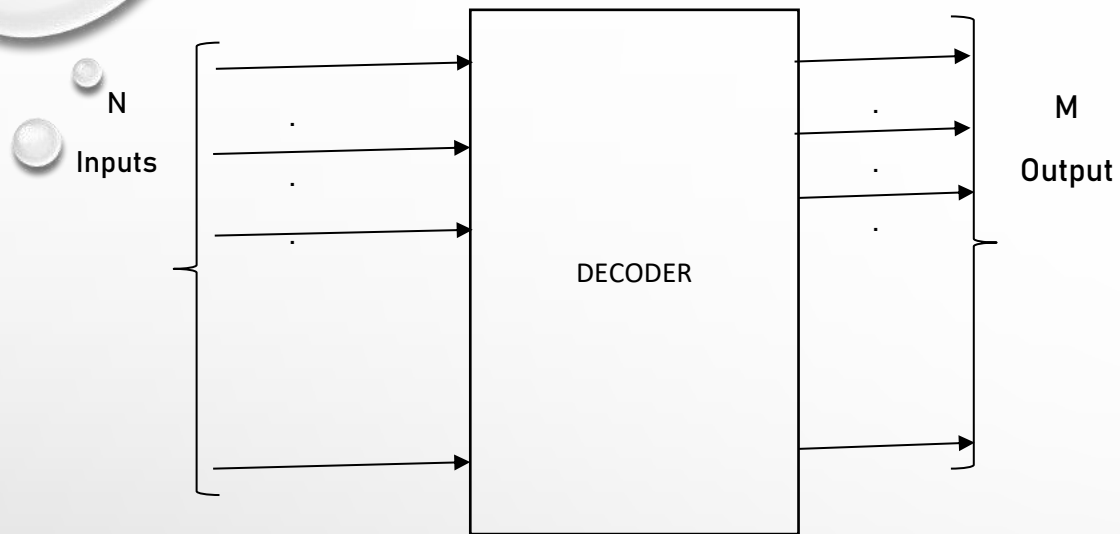
EXAMPLE OF PROCESS STATEMENT USING CASE STATEMENT

```
LIBRARY ieee ;  
USE ieee.std_logic_1164.all ;
```

```
ENTITY mux2to1 IS  
    PORT (d0, d1, s : IN STD_LOGIC;  
          Z      : OUT STD_LOGIC);  
END mux2to1 ;
```

```
ARCHITECTURE Behaviour OF mux2to1 IS  
BEGIN  
    PROCESS (d0, d1, s )  
    BEGIN  
        CASE s IS  
            WHEN '0' =>  
                Z <= d0;  
            WHEN OTHERS =>  
                Z <= d1;  
        END CASE ;  
    END PROCESS ;  
END Behaviour ;
```

VHDL CODE FOR A DECODER CIRCUIT



a. Logic Symbol

A decoder is a logic unit that converts an N-bit binary input code into M output such that each output line will be activated for only one of the possible combinations of inputs. The decoder Table shows a 2 to 4 decoder

A	B	Q0	Q1	Q2	Q3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

b. Decoder Table

VHDL code that describes a process statement of a 2-to-4 binary decoder using a CASE statement

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY dec2to4 IS
    PORT ( w : IN  STD_LOGIC_VECTOR(1 DOWNTO 0) ;
          En : IN  STD_LOGIC ;
          y : OUT STD_LOGIC_VECTOR(0 TO 3) ) ;
END dec2to4 ;

ARCHITECTURE Behavior OF dec2to4 IS
BEGIN
    PROCESS ( w, En )
    BEGIN
        IF En = '1' THEN
            CASE w IS
                WHEN "00" =>
                    y <= "1000" ;
                WHEN "01" =>
                    y <= "0100" ;
                WHEN "10" =>
                    y <= "0010" ;
                WHEN OTHERS =>
                    y <= "0001" ;
            END CASE ;
        ELSE
            y <= "0000" ;
        END IF ;
    END PROCESS ;
END Behavior ;
```

General Form of LOOP STATEMENTS

```
[loop_label;]  
FOR variable_name IN range LOOP  
    statement ;  
    {statement ;}  
END LOOP [loop_label] ;
```

```
[loop_label;]  
WHILE boolean_expression LOOP  
    statement ;  
    {statement ;}  
END LOOP [loop_label] ;
```

ASSIGNMENT

Write a VHDL code to represent the months of the year using a CASE statement